

ПРАКТИЧЕСКАЯ 11: ВИЗУАЛИЗАЦИЯ И РАБОТА С БОЛЬШИМИ ДАННЫМИ

Ожидаемое время выполнения: 3 часа (1 час — создание графиков, 1 час — работа с большими данными, 1 час — анализ и оформление отчета).

Цель практики

Освоить методы создания интерактивных графиков в Python, R.

Задачи практики

1. Создать интерактивные графики:

- Python: использование библиотеки plotly для построения графиков с возможностью взаимодействия (зум, фильтры).
- R: построение интерактивных панелей с использованием пакета shiny.

2. Работа с большими данными:

- Использовать фильтры, агрегацию и другие инструменты для упрощения анализа.

3. Сравнить подходы работы с большими данными в Python (pandas, dask), R (data.table).

4. Оптимизировать обработку данных:

- Анализ производительности каждого инструмента при работе с большим объёмом данных.

Формат отчета

1. Введение:

- Цель и задачи работы.
- Краткое описание преимуществ интерактивной визуализации и особенностей работы с большими данными.

2. Результаты работы:

- Интерактивные графики:
- Код для Python (plotly) и R (shiny).
- Работа с большими данными:

- Таблицы и графики, иллюстрирующие обработку данных в Python, R.
- Описание шагов оптимизации.
- Сравнительная таблица производительности инструментов.

3. Заключение:

- Выводы о наиболее удобных и эффективных инструментах для работы с большими данными.
- Анализ преимуществ и недостатков интерактивных визуализаций в Python, R.

Дополнительные материалы к практической работе 11

Исходные данные: Большой набор данных (например, данные о транзакциях, логах пользователей, или погодных измерениях).

ПОШАГОВАЯ ИНСТРУКЦИЯ К ПРАКТИЧЕСКОЙ РАБОТЕ 11

ШАГ 1. ПОДГОТОВКА ДАННЫХ

1.1. Общие требования к данным

Объём данных: Рекомендуется использовать набор данных, который нельзя быстро обработать в памяти на слабом компьютере (например, несколько миллионов строк), чтобы была видна разница в производительности.

Структура: Табличный формат (CSV, Parquet, SQL-база и т.д.). Должно быть минимум несколько числовых и категориальных столбцов, чтобы строить различные виды графиков и агрегировать данные.

Чистота: При необходимости удалите пропуски, обработайте аномалии, либо используйте встроенные механизмы обработки пропусков (импьютация, фильтрация и т.д.).

1.2. Проверка корректности

Убедитесь, что данные доступны для чтения как Python, так и R (например, формат CSV).

Для тестирования больших данных можно сгенерировать или скачать общедоступные датасеты (Kaggle, Open Data и т.д.).

ШАГ 2. ИНТЕРАКТИВНАЯ ВИЗУАЛИЗАЦИЯ

2.1. Python (Plotly)

Установка и импорт библиотек

```
# pip install plotly
import plotly.express as px
import plotly.graph_objects as go
import pandas as pd
```

Загрузка данных

```
df = pd.read_csv('big_data.csv')
print(df.head())
print(df.info())
```

Убедитесь, что все столбцы корректно считаны (числовые признаки, даты, категориальные переменные и т.д.).

Создание базового интерактивного графика

```
# Предположим, что у нас есть столбцы: 'category', 'value'
fig = px.bar(df, x='category', y='value', title="Пример
интерактивного графика")
fig.show()
```

Plotly график по умолчанию интерактивный: можно наводить на точки, зумировать, панорамировать.

Для расширенных возможностей (фильтры, анимации), можно использовать срезы (`df[df['category'] == 'A']`), группировки и агрегаты (`df.groupby(...).sum()`) перед построением.

Для анимации (time-based) использовать параметр `animation_frame`.

2.2. R (Shiny)

Установка и базовая структура приложения Shiny

```
# install.packages("shiny")
library(shiny)
```

```
# Минимальный шаблон
ui <- fluidPage(
  titlePanel("Пример Shiny-приложения"),
  sidebarLayout(
    sidebarPanel(
      # Элементы управления (input)
    ),
    mainPanel(
      # Вывод графиков (output)
      plotOutput("distPlot")
    )
  )
)

server <- function(input, output) {
  # Загрузка данных (или загрузка до ui/server)
  df <- read.csv("big_data.csv")

  output$distPlot <- renderPlot({
    # Рисуем базовый график
    hist(df$value, main="Распределение value", xlab="value")
  })
}

shinyApp(ui = ui, server = server)
```

Запуск через Run App в RStudio или shinyApp(ui = ui, server = server).

Добавление интерактивных элементов

```
ui <- fluidPage(
  titlePanel("Анализ больших данных - Shiny"),
  sidebarLayout(
    sidebarPanel(
      selectInput("categoryInput", "Выберите категорию:",
        choices = unique(df$category))
    ),
    mainPanel(
      plotOutput("plotCat")
    )
  )
)

server <- function(input, output) {
  df <- read.csv("big_data.csv")

  output$plotCat <- renderPlot({
    # Фильтруем по выбранной категории
    df_filtered <- subset(df, category == input$categoryInput)
    barplot(df_filtered$value, main=paste("Значения для
категории", input$categoryInput))
  })
}
```

```
shinyApp(ui = ui, server = server)
```

Можно добавлять селекторы (`selectInput`), слайдеры (`sliderInput`), чтобы пользователь выбирал фильтр или агрегат, а график перерисовывался.

ШАГ 3. РАБОТА С БОЛЬШИМИ ДАННЫМИ

3.1. Python (pandas, dask)

Профилирование и фильтрация:

- `pandas`: хороший вариант для данных до нескольких миллионов строк.
- `dask`: для ещё больших данных, разделённых на чанки.

Pandas

```
import pandas as pd

# Считываем только нужные столбцы / строки
df = pd.read_csv('big_data.csv', usecols=['col1', 'col2'],
nrows=10_000_000)
```

Dask

```
import dask.dataframe as dd

dd_df = dd.read_csv('big_data.csv')
# Вычисления "ленивые", реальный расчёт при вызове .compute()
dd_df.groupby('category')['value'].mean().compute()
```

Агрегация и уменьшение объёма очень полезна и реализовать ее можно следующим вариантом:

- Суммирование, среднее, медиана, `count` — для анализа без необходимости загружать весь датасет в оперативную память.
- Иногда имеет смысл использовать базы данных (SQL) и лишь вынимать необходимый срез.

3.2. R (data.table)

Установка и использование data.table

```
# install.packages("data.table")
library(data.table)

dt <- fread("big_data.csv")
# fread() быстрее считывает, чем base read.csv

# Пример группировки:
dt[, .(mean_value = mean(value, na.rm=TRUE)), by=category]
```

data.table имеет «lazy»-подобное поведение, но суть в том, что она более эффективна в оперативной памяти, чем data.frame

Агрегация и фильтрация

```
# Фильтрация
subset_dt <- dt[value > 100 & category == "A"]

# Агрегация
agg_dt <- dt[, .(sum_val = sum(value)), by=.(category, region)]
```

ШАГ 4. ОПТИМИЗАЦИЯ И АНАЛИЗ ПРОИЗВОДИТЕЛЬНОСТИ

Замеры времени:

В Python — использовать модуль time, timeit, или встроенные в jupyter магии (%time).

В R — функции system.time().

Сравнительные тесты:

Проведите несколько операций (группировка, фильтрация, соединение) и измерьте время исполнения в pandas/dask/data.table.

Оптимизации:

Сокращение числа столбцов, чтение в формате Parquet (сжатие),

использование бинарных форматов.

Разбиение набора данных на части (chunking) и последовательная обработка.

КЛЮЧЕВЫЕ МОМЕНТЫ ДЛЯ УСПЕШНОГО ВЫПОЛНЕНИЯ ПРАКТИЧЕСКОЙ РАБОТЫ

Демонстрация интерактивности:

Покажите, как пользователь может менять параметры/фильтры и мгновенно видеть перерисованный график (особенно в Shiny).

Большие данные:

На практике используйте хотя бы несколько миллионов строк, чтобы разница в производительности была ощутима.

Оптимизация:

Продемонстрируйте разные подходы (встроенные возможности pandas vs. dask, или data.frame vs. data.table).

Сравните результаты и укажите свои наблюдения.

Визуальное оформление отчёта:

Графики, таблицы, скриншоты крайне важны для наглядного представления результатов.

Данные шаги помогут вам освоить основы интерактивной визуализации в Python и R, а также познакомят с базовыми приёмами работы с большими объёмами данных, методами оптимизации и анализом производительности в разных инструментах.